

Effective vs. Exhaustive Testing

Why you can never test everything — and what to test instead.

UNIVERSITY QUESTION · MU — DEC. 16, DEC. 18

“Explain effective and exhaustive software testing.”

THE CORE PROBLEM

“When are we done with testing? How do we know we have tested enough?”

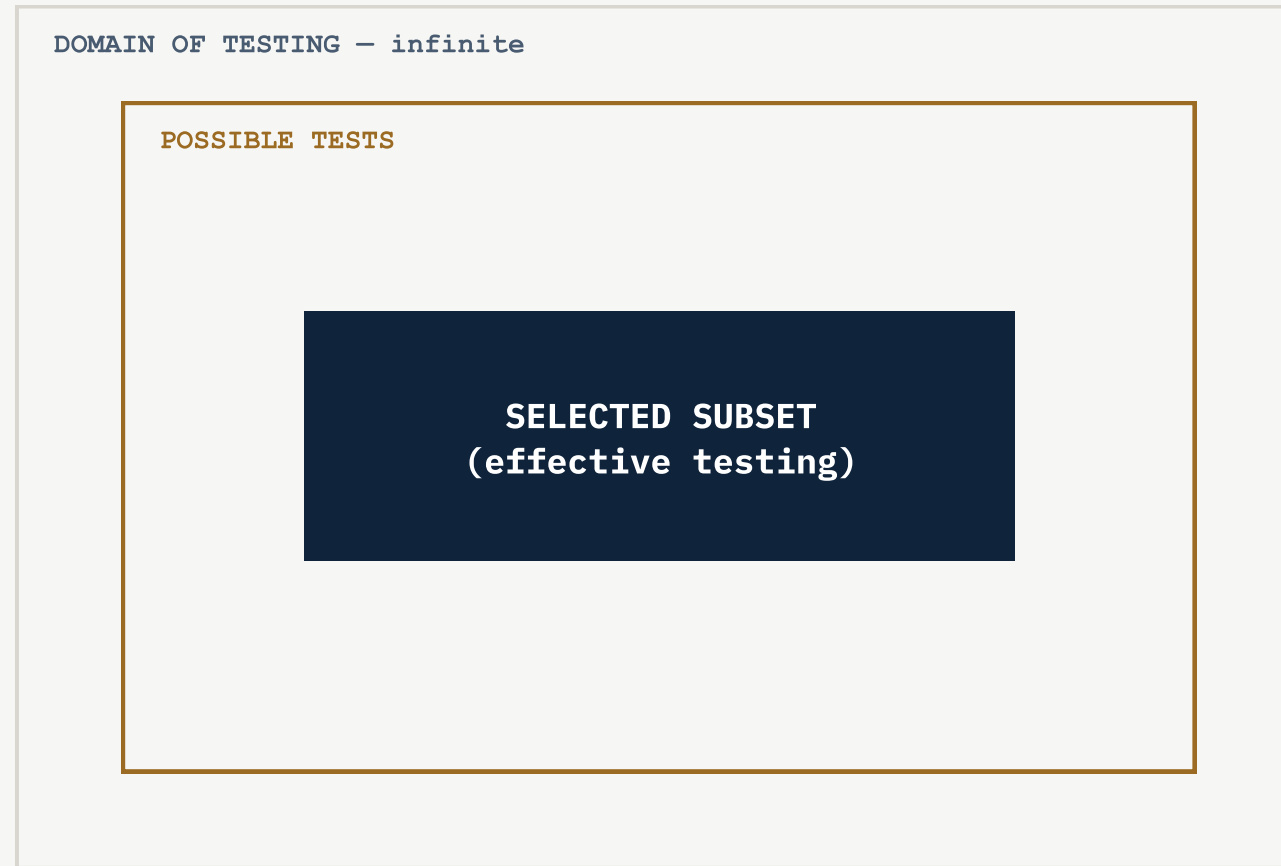
Exhaustive, or complete, software testing means every statement in the program — and every possible path, in every possible combination of data — must be executed. It sounds thorough. It is also, as we're about to prove, impossible.

So we concentrate on EFFECTIVE testing — efficient techniques that test what matters, within the resources we actually have.

The Testing Domain

Testing is a domain of possible tests — and the domain is infinite.

As the domain grows, we can never test every possible combination — only a chosen subset, selected for what matters most in this environment.



Effective vs. Exhaustive Testing

Effective Testing	Exhaustive Testing
Efficient techniques test important features within constrained resources.	Every statement, every path, every combination of data must be executed.
A practical method.	Not possible to perform completely.
Feasible: checks reliability, tests each phase, uses constrained resources.	Not feasible: deadlines, endless outputs, timing constraints, too many environments.
Cost effective.	Not cost effective.
Less complex, less time consuming.	Complex and time consuming.
Critical test cases are addressed first.	Attempts to cover every test case.

The Combinatorics Problem

Even the simplest program has an input domain too large to fully test.

Take the simplest possible example: a program that adds two two-digit numbers, each ranging from -99 to 99.

$$199 \times 199 = 39,601$$

Widen the range from two digits to four, and the count jumps to 399,960,001 combinations — for a program most people would call “trivial.”

DEFINITION

Positive Testing

Testing software with valid data — always performed keeping the valid range or limits of the test data in view.

It seems like we could test every valid input — until the arithmetic above shows just how fast that possibility explodes.

The Input Domain, in Four Parts

1

Valid Inputs

Positive testing — the valid range of test data.

2

Invalid Inputs

Negative testing — tries to break the software on purpose.

3

Edited Inputs

Unexpected input events: stray spaces, buffer overflows.

4

Race Condition Inputs

Timing variation between two or more input events.

Invalid Inputs — Trying to Break It

When we test software with invalid data, it's called negative testing — performed to make sure the software behaves properly when it's fed data it was never meant to receive. The important thing is the program's behaviour, not just whether it “works.”

A set of invalid inputs, just like valid ones, is too large to test in full.

FIVE WAYS THE ADDITION EXAMPLE CAN BREAK

- 1 Numbers out of range
- 2 A combination of alphabets and digits
- 3 A combination of all alphabets
- 4 A combination of control characters
- 5 Any other key on the keyboard

What a User Might Do – On Purpose or By Accident

03 · EDITED INPUTS

If a user can edit input as they type it, unexpected events happen — and behaviour can't be fully predicted.

EXAMPLE

Invisible extra spaces can silently break a program. Or: a user presses a number key, then Backspace repeatedly, then another number and Enter — the input buffer overflows, and the system crashes.

User behaviour can't be judged in advance — which is exactly why edited inputs are never tested completely.

04 · RACE CONDITION INPUTS

Timing variation between two or more inputs limits testing — by design, A precedes B in most cases.

A → B the expected order, in most cases

B → A rare, restricted — the race condition

The program usually fails because the rare “B before A” case was never accounted for — a race condition bug. These are especially common in multiprocessing and interactive systems, and are among the least-tested inputs.

The Intel Pentium FDIV Bug

What happens when the input domain really is too large to test — and nobody catches the five missing entries in time.

In 1994, Intel's brand-new Pentium processor shipped with a flaw in its floating-point division circuitry. A lookup table used by the division algorithm was supposed to contain 1,066 values — but five entries were missing. For most calculations, this made no difference at all. But for certain, rare pairs of numbers, the chip returned a wrong answer.

THE INPUT DOMAIN, AGAIN

Byte magazine estimated only 1 in 9 billion random floating-point divisions would even hit the flaw — exactly the combinatorial explosion we just saw with two-digit addition.

5

missing entries out of 1,066 in a single lookup table

RECALL COST

\$475M

DISCOVERED BY

A math professor

ROOT CAUSE

An input domain too vast to fully verify

In June 1994, mathematician Thomas Nicely noticed his prime-number calculations were subtly wrong — but only on his one Pentium-based machine. By October, he'd ruled out every other explanation and reported the flaw directly to Intel.

Intel initially estimated the error would occur once every 27,000 years of typical use. IBM's own analysis put it at once every 24 days — and halted Pentium-based PC sales entirely.

Four Transferable Lessons

LESSON 01

“Rare” is not “never”

One in nine billion still means thousands of scientists and engineers hit the bug eventually — rare events happen at scale.

LESSON 02

You can't test the whole domain – so choose wisely

Intel's own internal testing had found the bug in May 1994; the failure was in prioritizing and escalating it, not just finding it.

LESSON 03

Silence is its own decision

Intel discovered the flaw internally months before Nicely, and chose not to disclose it — that choice became the bigger story.

LESSON 04

Precision problems compound at scale

A tiny, rare arithmetic error is harmless in a spreadsheet, but potentially serious in scientific, financial, or engineering calculations run millions of times.